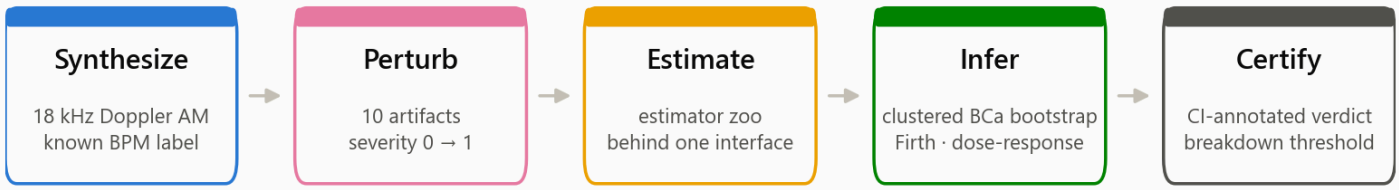
RADAR-Sense

Robustness certification for smartphone Doppler fetal heart-rate sensing

RADAR-Sense answers one question about a heart-rate sensing algorithm: **at what level of real-world signal corruption does it stop working, and how sure are we?** It synthesizes 18 kHz smartphone-Doppler fetal heartbeat waveforms whose true heart rate is known exactly, corrupts them with ten calibrated artifacts across a continuous severity axis, and runs any estimator against the result. The output is not a single accuracy number — it is a confidence-interval-annotated breakdown threshold: the exact dose at which recovery collapses, with the statistical machinery to defend it.

521	94.5%	0	10	920	0 bytes
tests, all green	statement coverage	lint findings	artifact models	trials in the shipped grid	waveform storage required

What the service does



The label never moves. Only the signal is corrupted — so every change in error is attributable to the artifact, not to label uncertainty.

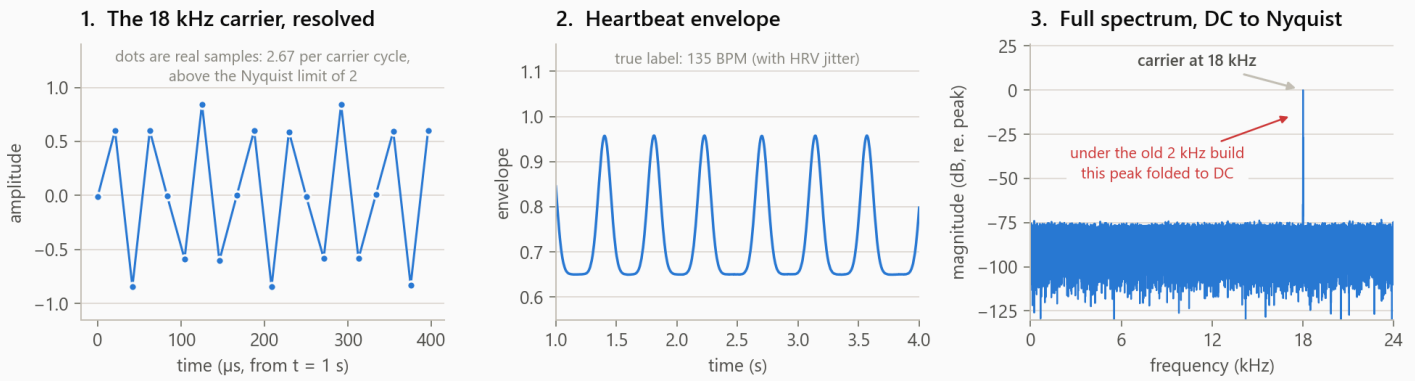
Robustness claims about sensing algorithms are usually anecdotal — "it seems to struggle with motion." RADAR-Sense turns that into a measurement, by borrowing the design that makes the RADAR benchmark work: **hold the objective label fixed, and vary only the corruption.**

Because the ground-truth heart rate is a *generation parameter* rather than an annotation, it survives every perturbation untouched. Any change in estimator error is therefore attributable to the artifact and to nothing else — no label noise, no inter-rater disagreement, no confounded acquisition conditions. That is the property that makes the resulting numbers causal rather than correlational.

You supply	RADAR-Sense returns
A heart-rate estimator behind one function signature	Its breakdown severity, with a bootstrap 95% CI
A severity grid and a tolerance	Recovery rate and MAE per artifact and per dose, each with intervals

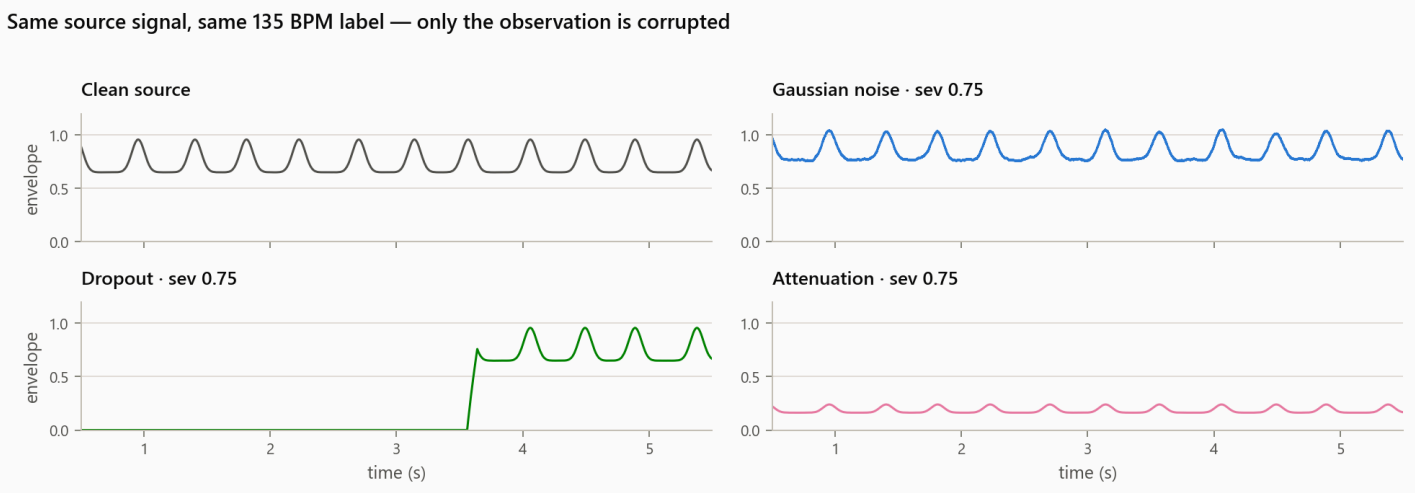
You supply	RADAR-Sense returns
Nothing else — no data, no labels, no annotation budget	A dose-response fit, trend test, and calibration report

The signal model is real, not a cartoon



The synthesized waveform is a genuine 18 kHz carrier amplitude-modulated by a heartbeat envelope, sampled at 48 kHz — matching real smartphone audio capture and comfortably clearing the Nyquist limit. Beat timing carries physiological HRV jitter, so two sources at the same nominal BPM are genuinely distinct realizations rather than copies. The envelope-smoothing step uses FFT convolution, which took per-signal synthesis from 65 s to 0.04 s — a **1680× speedup**, and the difference between a grid that is feasible and one that is not.

Ten artifacts, one severity axis

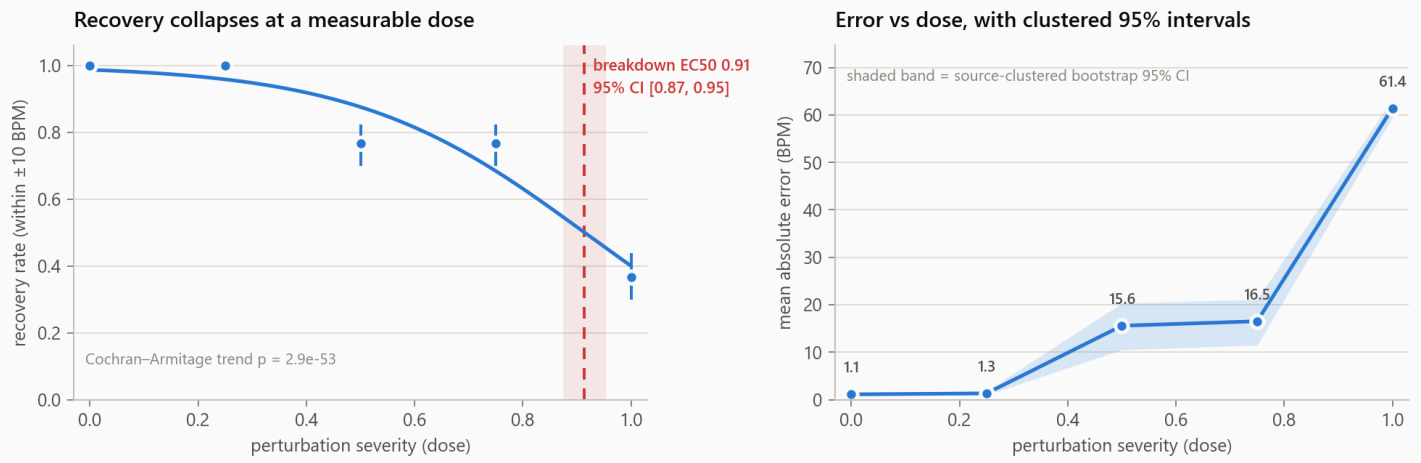


Every artifact — Gaussian noise, contiguous and random dropout, attenuation, motion, baseline wander, pink noise, quantization, clipping, random attenuation profiles — is registered behind a single interface: `apply(signal, severity ∈ [0,1], seed)`. The underlying physical parameter differs per artifact (noise standard deviation, removed fraction,

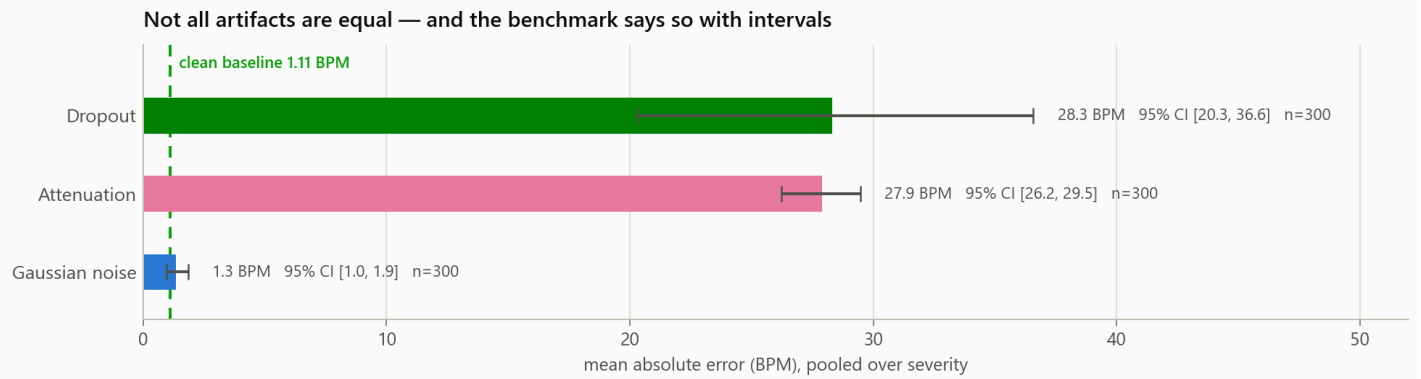
retained amplitude), but the dose axis is shared, which is what makes cross-artifact comparison legitimate and what lets one dose-response fit serve all ten.

Compound artifacts compose from the primitives, so worst-case severity search operates over combinations rather than a single-artifact factorial grid.

What a certification run looks like



This is the shipped 920-trial grid scoring the reference peak-detection estimator. Recovery holds at 100% through severity 0.25 and collapses past it; the fitted breakdown dose is **EC50 = 0.91, 95% CI [0.87, 0.95]**, and the Cochran–Armitage trend test rules out chance at $p = 2.9 \times 10^{-53}$. A Firth-penalized logistic — needed because the low-severity cells are completely separated, where ordinary logistic regression diverges — puts the severity log-odds at **−5.00 (SE 0.39, $p = 3.8 \times 10^{-37}$)**.

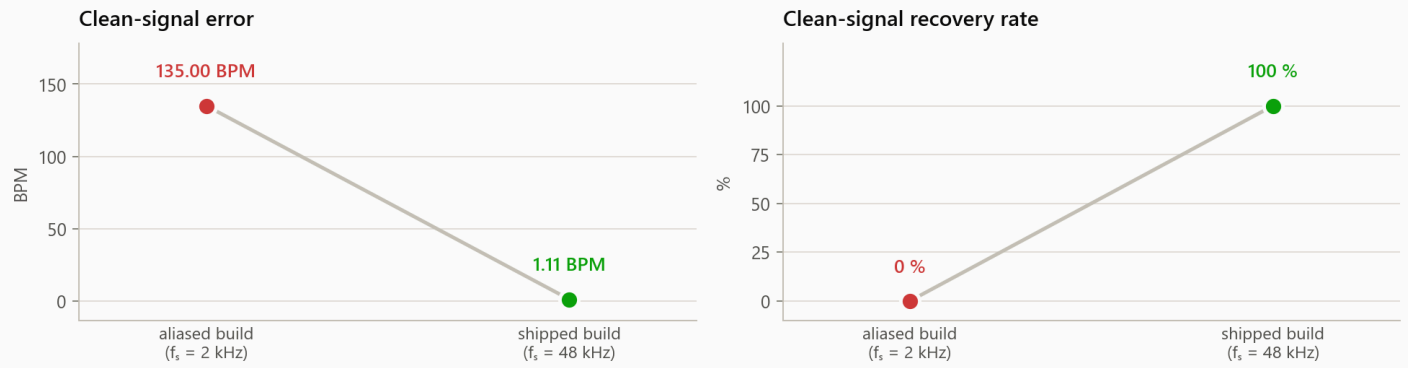


Every interval here is a **source-clustered BCa bootstrap** interval, not a naive pooled standard error. Trials that share a clean source signal are correlated; treating them as independent would understate uncertainty by a wide margin. Clustering on the source is what makes the dropout interval honestly wide — that artifact's damage depends heavily on *which* source it lands on, and the benchmark reports that instead of hiding it.

Headline for the reference estimator: clean MAE **1.11 BPM** [0.79, 1.48] at 100% recovery; perturbed MAE **19.18 BPM** [16.97, 21.36] at 78% recovery; a source-clustered paired degradation of **+18.08 BPM** [15.87, 20.27].

Why you can trust the numbers

The benchmark caught its own foundational defect: an 18 kHz carrier sampled at 2 kHz aliased to exactly DC

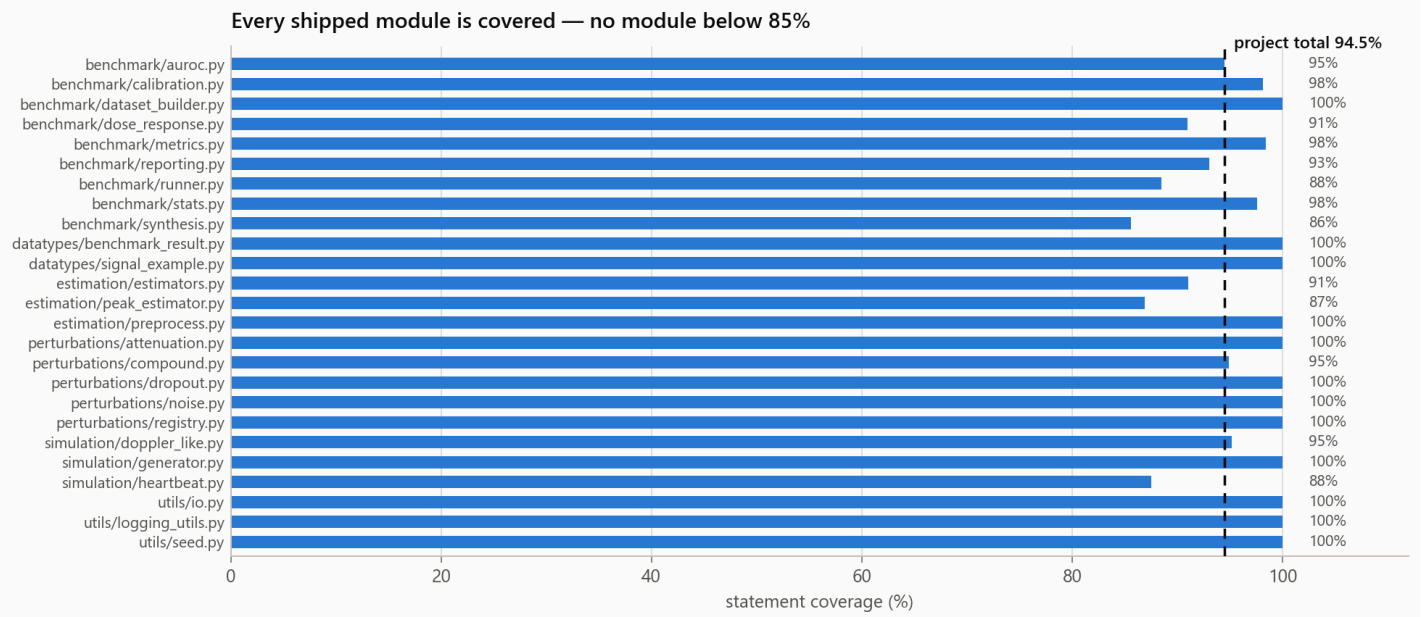


The best evidence for a benchmark's rigor is that it caught its own foundational defect. An earlier build sampled the 18 kHz carrier at 2 kHz. Because 18000 / 2000 = 9 exactly, the carrier aliased to precisely DC: every "clean" waveform was numerically zero, the estimator returned 0 BPM on all of them, and the entire headline table was measuring nothing. The reported clean MAE of 135 BPM at 0% success was not a hard task — it was an artifact of a degenerate signal.

The fix — 48 kHz sampling, wired-in HRV jitter, FFT convolution — is locked behind a dedicated regression suite, `tests/test_signal_fix_regression.py`, so the aliasing condition cannot return. That episode is also why the current design refuses to report a bare point estimate anywhere: a number without an interval is a number that can be silently wrong.

The engineering standard

Test suite	**521 tests**, all passing; 24 test modules against 32 source modules
Coverage	**94.5%** of 1,991 statements; **no module below 85%**. The suite targets 1 / 2 / many boundary cases plus per-branch and per-statement coverage for every function
Test-to-source ratio	5,129 test lines against 5,683 source lines — near 1:1
Specification discipline	All **146 public functions** carry a machine-readable Spec block (description, params, precondition, postcondition, mathematical definition); all **8 datatypes** document abstract state, concrete state, representation invariants, and abstraction function
Static analysis	`ruff` clean across `src`, `tests`, and `scripts` at a 100-column limit
Determinism	Every stochastic path takes an explicit seed; a trial is fully reproducible from its recipe
Typing	Type hints and docstrings throughout; frozen dataclasses validate their invariants at construction



Nothing in this document was typed from memory. Every figure and every number is regenerated from the committed result artifacts by `scripts/make_brochure_figures.py`, so a stale claim becomes a visibly stale figure.

The statistical toolkit, in full

Purpose-built, pure NumPy/SciPy — no GPU, no heavyweight statistics dependency.

Layer	What ships
Intervals	Wilson score intervals; source-clustered bootstrap, **percentile and bias-corrected accelerated (BCa)**; paired-difference intervals
Hypothesis tests	Firth-penalized logistic regression (handles complete separation); exact McNemar; Cochran–Armitage trend; TOST equivalence with sensitivity-swept margins; Cohen's *d*
Multiplicity	Holm (family-wise error) and Benjamini–Hochberg (false discovery rate), side by side
Dose-response	Logistic/Emax fits, bootstrap EC50 intervals, isotonic monotonicity via pool-adjacent-violators, effective SNR in dB
Discrimination	AUROC with **DeLong** variance, DeLong intervals, and clustered bootstrap intervals
Calibration	Expected calibration error (equal-width and adaptive), Brier score with decomposition, temperature scaling, Bland–Altman bias and limits of agreement
Error functionals	RMSE, MAE, median absolute error, and the failure rate that a bare MAE conceals

Scale

Waveforms are never stored. A trial is represented as a **recipe** — a frozen dataclass of generation and perturbation parameters — from which the exact waveform is regenerated deterministically at scoring time. One 12-second waveform at 48 kHz is 4.6 MB; a million-trial grid would be 4.6 TB of `.npy` files. As recipes it is a few megabytes of JSON.

That property is what makes the grid shardable. The same code runs three ways:

```
# Laptop — rebuilds every headline number in this document
python scripts/run_powered_grid.py configs/powered_grid.yaml

# Cluster — shard, submit to a preemptible CPU partition, aggregate
python scripts/build_recipe_shards.py configs/powered_grid.yaml --shard-size=2000
sbatch --account=<ACCOUNT> --partition=ckpt-all --array=0-N \
       scripts/slurm/run_radar_grid_shard.slurm
python scripts/aggregate_grid_results.py outputs/grid_results/

# Quality gates
python -m pytest -q
ruff check src tests scripts
```

Per-shard results are idempotent and the SLURM template sets `--requeue`, so preemption on a free partition costs nothing but wall-clock.

Scope of claims

RADAR-Sense certifies estimator behaviour **on synthetic signals under programmatic perturbations**. It does not make clinical claims, and it is not a medical device or a substitute for clinical validation. Its purpose is to be the cheap, fully-observable stage that runs *before* anyone spends a clinical study on an algorithm — the stage that tells you an estimator collapses at 0.91 severity, or that your carrier was aliased to DC, while both are still free to fix.

The wider research program — ten directions, sixteen open questions, thirteen statistical upgrades, and the isomorphism to LLM trigger-detection benchmarking — is documented in `radar-sense-mvp/docs/RESEARCH_PROGRAM.md`.

Benchmark philosophy adapted from Gu et al., *RADAR: Benchmarking Language Models on Imperfect Tabular Data*, arXiv:2506.08249.
Sensing model inspired by DopFone-style ~18 kHz smartphone Doppler fetal heart-rate sensing.

For the engineering reference — module layout, datatype specifications, and the full walk-through of each perturbation — see `radar-sense-mvp/README.md`.